

3일차(2)

날짜	2026년 7월 4일
상태	완료
# 시간	3
요약	[실습] Rekognition API 실습(이어서) Amazon Textract [실습] Textract 데모 + Textract API 실습

Amazon Textract

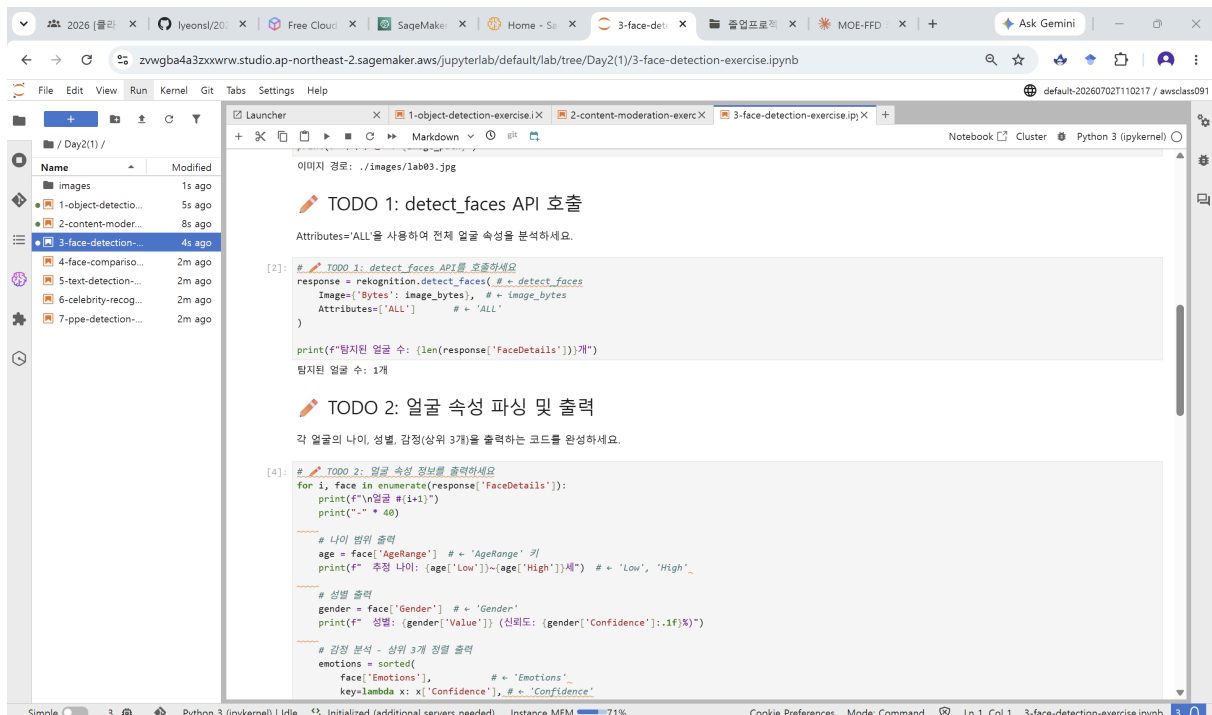
[attachment:99919060-a30b-4ce6-898d-55bd83672954:교안_06_AWS_Textract.pdf](#)

Amazon Textract : 문서 텍스트/테이블 분석 및 추출해주는 AWS 관리형 문서 AI 서비스

[실습] Rekognition API 실습(이어서)얼굴 탐지 (Face Detection)

```
rekognition.detect_faces(Image={'Bytes': image_bytes}, Attributes=[])
```

- 이미지에서 얼굴을 탐지하고 다양한 속성을 반환



`response['FaceDetails']` — 각 얼굴의 상세 정보 리스트

얼굴 비교 (Face Comparison)

```
rekognition.compare_faces(  
    SourceImage={'Bytes': source_bytes},  
    TargetImage={'Bytes': target_bytes},  
    SimilarityThreshold=  
)
```

- 소스(기준) 이미지의 얼굴과 타겟 이미지의 얼굴을 비교

이 이미지 로딩 완료

TODO 1: compare_faces API 호출

두 이미지의 얼굴을 비교하세요. SimilarityThreshold는 80으로 설정하세요.

```
[2]: # TODO 1: compare_faces API를 호출하세요  
response = rekognition.compare_faces( # = compare_faces  
    SourceImage={'Bytes': source_bytes}, # = source_bytes  
    TargetImage={'Bytes': target_bytes}, # = target_bytes  
    SimilarityThreshold=80 # = 80  
)  
  
matches = response['FaceMatches']  
unmatched = response['UnmatchedFaces']  
print(f"#매칭된 얼굴: {len(matches)}개")  
print(f"#매칭 안 된 얼굴: {len(unmatched)}개")  
  
매칭된 얼굴: 0개  
매칭 안 된 얼굴: 1개
```

TODO 2: 비교 결과 출력

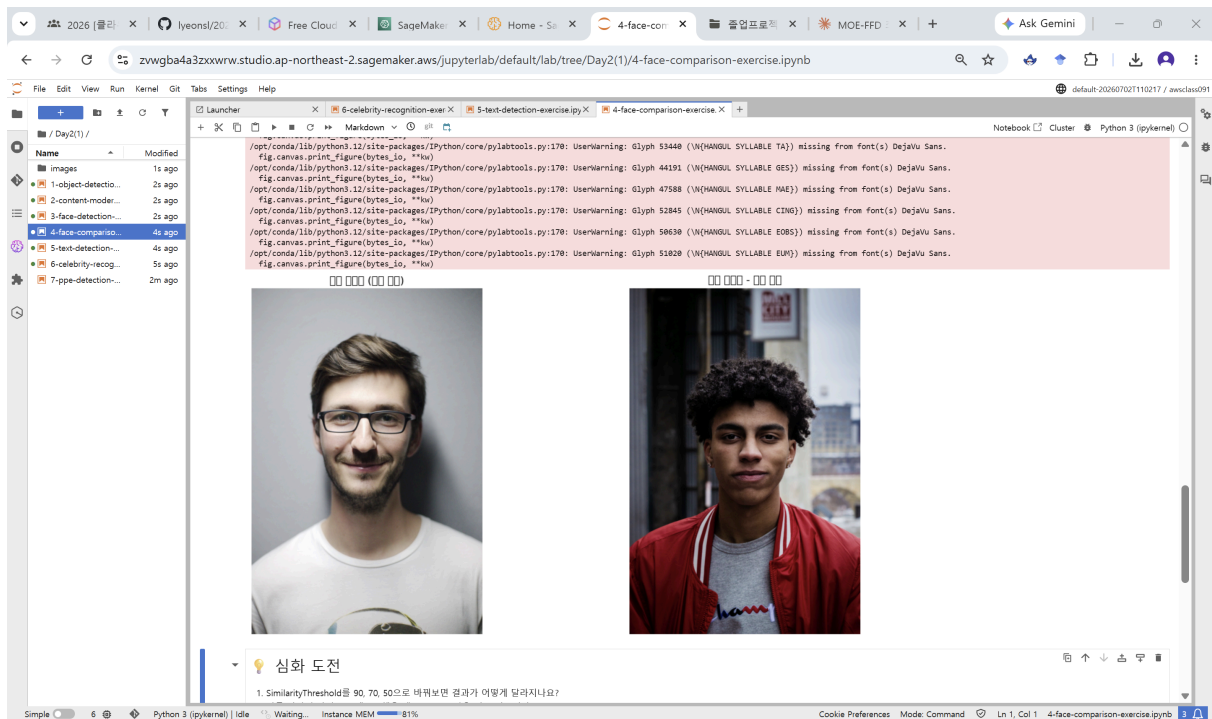
매칭된 얼굴의 유사도와 위치 정보를 출력하세요.

```
[3]: # TODO 2: 비교 결과를 출력하세요  
if len(matches) > 0:  
    print("\n✅ 동일 인물로 판정된 얼굴:")  
    print("-" * 40)  
    for i, match in enumerate(matches):  
        similarity = match['Similarity'] # = 'Similarity'  
        face = match['Face'] # = 'Face'  
        confidence = face['Confidence'] # = 'Confidence'  
        print(f"# 얼굴 #{i+1}: 유사도: {similarity:.1f}%, 위치 신뢰도: {confidence:.1f}%")  
else:  
    print("\n❌ 동일 인물이 발견되지 않았습니다.")  
  
❌ 동일 인물이 발견되지 않았습니다.
```

TODO 3: 두 이미지 나란히 시각화

소스와 타겟 이미지를 나란히 표시하고, 결과를 제목에 반영하세요.

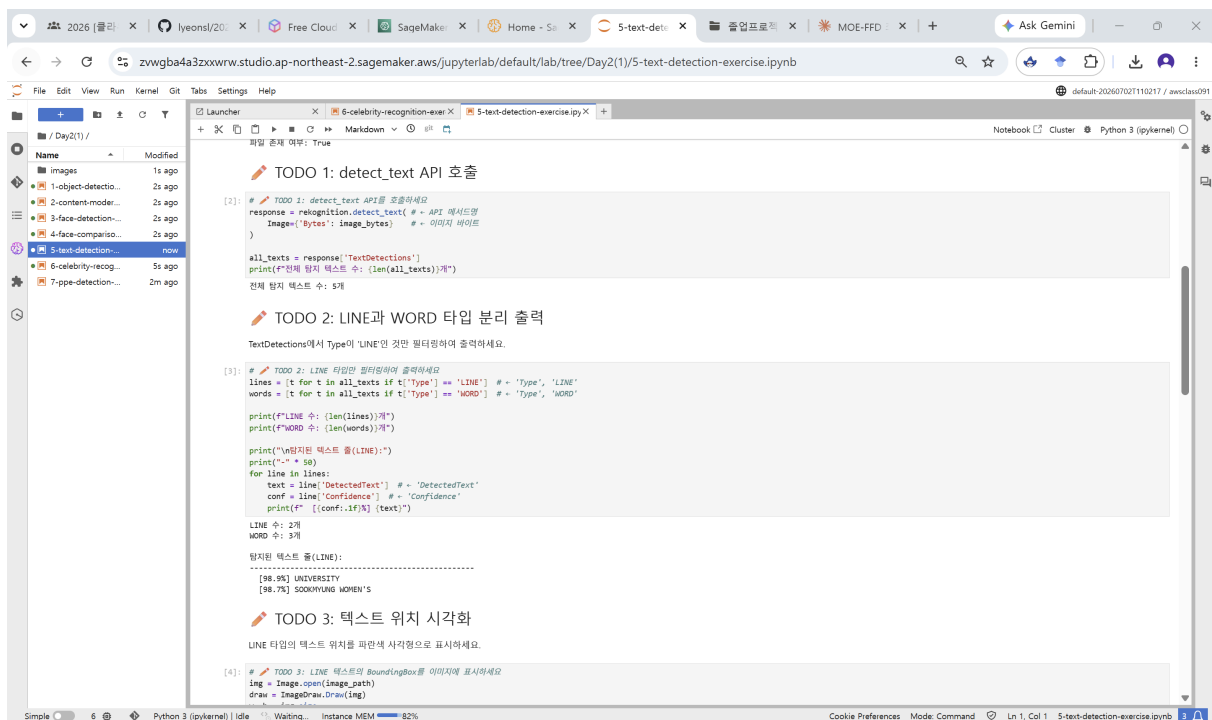
```
[ ]: # TODO 3: 두 이미지를 나란히 표시하세요
```



텍스트 탐지 (Text Detection)

```
rekognition.detect_text(Image={'Bytes': image_bytes})
```

- 이미지에서 텍스트를 감지



유명인 인식 (Celebrity Recognition)

```
rekognition.recognize_celebrities(Image={'Bytes': image_bytes})
```

- 이미지에서 유명인을 탐지

```

image_bytes = load_image_bytes(image_path)
print(f"이미지 경로: {image_path}")
이미지 경로: ./images/lab06.jpg

TODO 1: recognize_celebrities API 전체 작성
이 실습에서는 API 호출부터 결과 출력까지 직접 작성합니다.
참고: 이전 실습들의 패턴을 활용하세요!

[2]: # TODO 1: API를 호출하고 결과를 변수에 저장하세요
response = rekognition.recognize_celebrities(
    Image={'Bytes': image_bytes}) # - 전체 작성

celebrities = response['CelebrityFaces'] # - 'CelebrityFaces'
unrecognized = response['UnrecognizedFaces'] # - 'UnrecognizedFaces'

print(f"인식된 유명인: {len(celebrities)}명")
print(f"인식 못한 얼굴: {len(unrecognized)}명")

인식된 유명인: 1명
인식 못한 얼굴: 0개

TODO 2: 유명인 정보 출력

[3]: # TODO 2: 각 유명인의 이름과 신뢰도, URL을 출력하세요
print("인식된 유명인:")
print("-" * 50)
for celeb in celebrities:
    name = celeb['Name'] # - 'Name'
    conf = celeb['MatchConfidence'] # - 'MatchConfidence'
    urls = celeb.get('Urls', []) # - 'Urls'
    print(f"이름: {name}")
    print(f"신뢰도: {conf:.1f}%")
    if urls:
        print(f"참고 URL: {urls[0]}")
    print()

인식된 유명인:
이름: Park Bo-gum
신뢰도: 100.0%
참고 URL: www.wikidata.org/wiki/Q15977222
  
```

```

TODO 3: 유명인 얼굴에 이름 레이블 표시 (직접 작성)
이전 실습 패턴을 참고하여 인식된 유명인의 얼굴에 이름을 표시하는 코드를 지금부터 작성하세요.

[4]: # TODO 3: 유명인 얼굴 시각화 코드를 지금부터 작성하세요

# 시각화 코드를 작성하세요
img = Image.open(image_path)
draw = ImageDraw.Draw(img)
w, h = img.size
for celeb in celebrities:
    box = celeb['Face']["BoundingBox"] # <- 원본: 이전 실습(3, 4, 5)의 BoundingBox 처리 패턴을 참고하세요
    left = box['left'] * w
    top = box['top'] * h
    right = box['right'] * w
    bottom = box['bottom'] * h
    draw.rectangle([left, top, right, bottom], outline='gold', width=3)
    draw.text((left + box['left'], top + box['top'], right + box['right'], bottom), celeb['Name'], fill='gold')
plt.figure(figsize=(10, 8))
plt.imshow(img)
plt.axis('off')
plt.title('Celebrity Recognition 결과')
plt.show()

/opt/conda/lib/python3.12/site-packages/Python/core/pylabtools.py:170: UserWarning: Glyph 44208 (U+HANJUL SYLLABLE GYEOL) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
/opt/conda/lib/python3.12/site-packages/Python/core/pylabtools.py:170: UserWarning: Glyph 44284 (U+HANJUL SYLLABLE GUA) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)

Celebrity Recognition
  
```

개인 보호 장구 탐지 (PPE Detection)

```
rekognition.detect_protective_equipment(
    Image={'Bytes': image_bytes},
```



```

SummarizationAttributes={
    'MinConfidence': ,
    'RequiredEquipmentTypes': ['HEAD_COVER', 'FACE_COVER', 'HAND_COVE
R']
}
)

```

- 이미지에서 보호장구 착용 여부를 분석

2026 [클라] x lyons/20 x Free Cloud x SageMaker x Home - S5 x 7-ppe-dete x 졸업프로젝 x MOE-FFD x + Ask Gemini

zwgba4a3zxwvw.studio.ap-northeast-2.sagemaker.aws/jupyterlab/default/lab/tree/Day2(1)/7-ppe-detection-exercise.ipynb

File Edit View Run Kernel Git Tabs Settings Help

Launcher 7-ppe-detection-exercise.ipynb Notebook Cluster Python 3 (pykernel)

TODO 1: detect_protective_equipment API 호출

필수 장비로 HEAD_COVER와 FACE_COVER를 요구하는 API 호출을 작성하세요.

```

[2]: # TODO 1: API를 호출하세요
response = recognition.detect_protective_equipment( # - API 테스트용
    Images={'Bytes': image_bytes}, # - 이미지 바이트
    SummarizationAttributes={
        'MinConfidence': 80, # - 최소 신뢰도 (80 권장)
        'RequiredEquipmentTypes': ['HEAD_COVER', 'FACE_COVER'] # - 'HEAD_COVER', 'FACE_COVER'
    }
)

persons = response['Persons']
print(f"검지된 인원: {len(persons)}명")

검지된 인원: 2명

```

TODO 2: 인원별 보호장구 착용 상태 출력

각 인원의 신체 부위별 보호장구 착용 여부를 출력하세요.

```

[3]: # TODO 2: 인원별 PPE 착용 현황을 출력하세요
for i, person in enumerate(persons):
    print(f"인원 #{i+1}")
    print("-" * 40)

    for body_part in person['BodyParts']: # - 'BodyParts'
        part_name = body_part['Name'] # - 'Name'
        equipments = body_part['EquipmentDetections'] # - 'EquipmentDetections'

        if equipments:
            for equip in equipments:
                etype = equip['Type'] # - 'Type'
                covers = equip['CoversBodyPart']['Value'] # - 'Value'
                conf = equip['Confidence']
                status = "✓ 착용" if covers else "✗ 미착용/불완전"
                print(f"  {part_name} : {etype} - {status} ({conf:.1f})")
            else:
                print(f"  {part_name} : 보호장구 없음")

인원 #1
-----
FACE: 보호장구 없음
LEFT_HAND: HAND_COVER - ✓ 착용 (89.9%)
HEAD: HEAD_COVER - ✓ 착용 (97.6%)

```

Simple 7 Python 3 (pykernel) Idle Initialized (additional servers needed) Instance MEM 85%

2026 [클라] x lyons/20 x Free Cloud x SageMaker x Home - S5 x 7-ppe-dete x 졸업프로젝 x MOE-FFD x + Ask Gemini

zwgba4a3zxwvw.studio.ap-northeast-2.sagemaker.aws/jupyterlab/default/lab/tree/Day2(1)/7-ppe-detection-exercise.ipynb

File Edit View Run Kernel Git Tabs Settings Help

Launcher 7-ppe-detection-exercise.ipynb Code Notebook Cluster Python 3 (pykernel)

FACE: 보호장구 없음
LEFT_HAND: HAND_COVER - ✓ 착용 (91.7%)
RIGHT_HAND: HAND_COVER - ✓ 착용 (52.7%)
HEAD: HEAD_COVER - ✓ 착용 (99.6%)

TODO 3: Summarization 결과로 컴플라이언스 판정

Summary를 활용하여 안전 준수율을 계산하세요.

```

[7]: # TODO 3: Summary 기반 컴플라이언스 판정
summary = response.get('Summary', {})
with_ppe = summary.get('PersonsWithRequiredEquipment', [])
without_ppe = summary.get('PersonsWithoutRequiredEquipment', [])
indeterminate = summary.get('PersonsIndeterminate', [])

print("\n\n 컴플라이언스 요약:")
print("-" * 40)
print(f"  안전 인원: {len(with_ppe)}명")
print(f"  미착용 인원: {len(without_ppe)}명")
print(f"  불명확 인원: {len(indeterminate)}명")

total = len(persons)
if total > 0:
    compliance_rate = len(with_ppe) / total * 100 # - 안전 인원 / 전체 인원 * 100
    print(f"\n 안전 준수율: {compliance_rate:.1f}%")
    if compliance_rate == 100:
        print("  🟢 전원 안전 기준 충족")
    else:
        print("  🟡 일부 인원 PPE 미착용 - 주의 요망")

컴플라이언스 요약:
-----
  안전 인원: 0명
  미착용 인원: 2명
  불명확 인원: 0명

안전 준수율: 0.0%
  🟡 일부 인원 PPE 미착용 - 주의 요망

```

TODO 4: PPE 착용/미착용 시각화

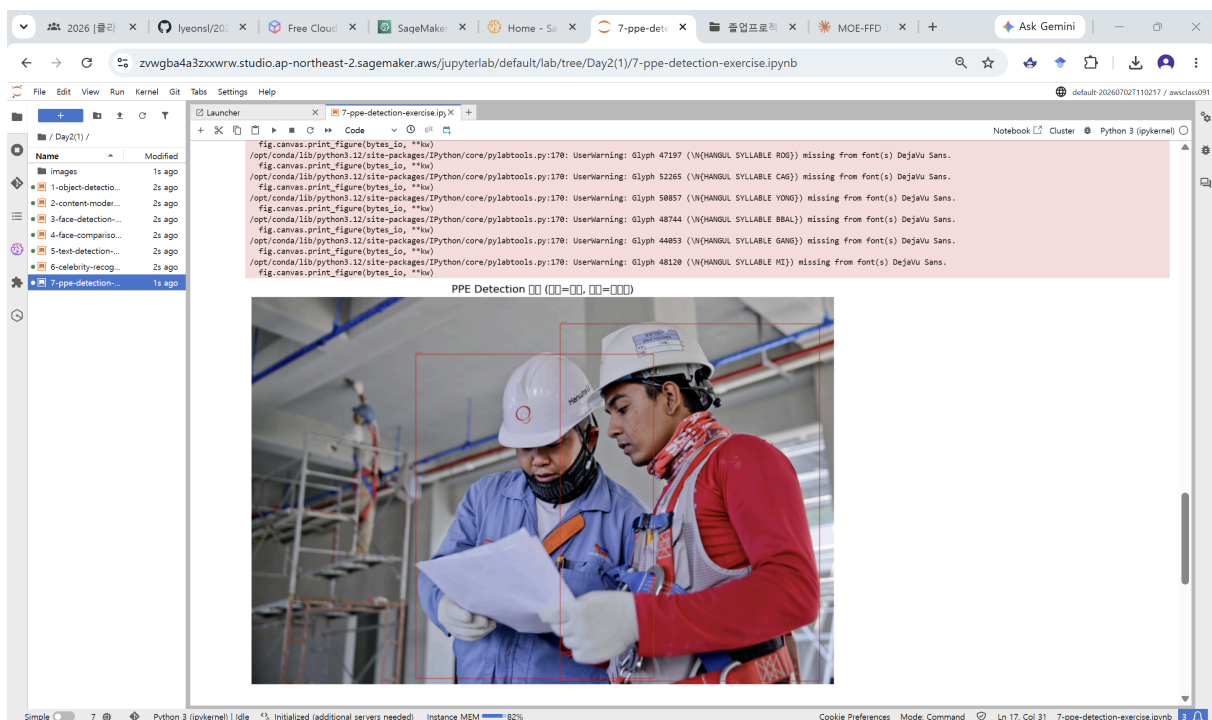
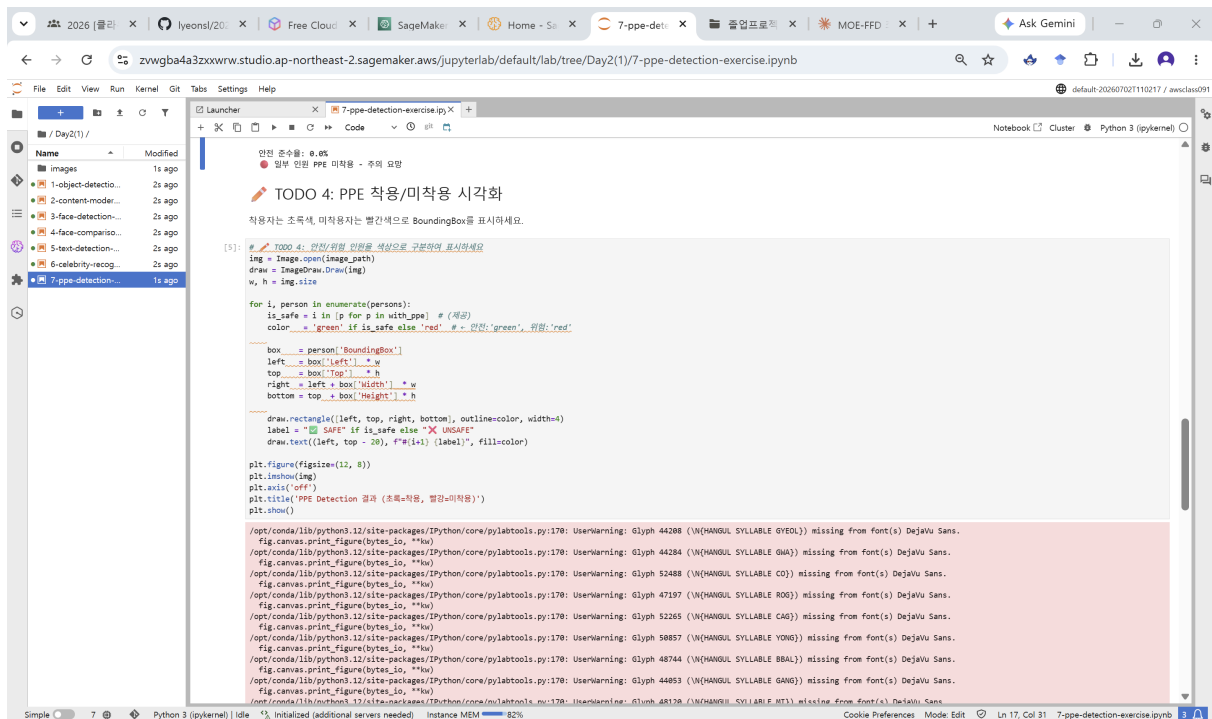
착용자는 초록색, 미착용자는 빨간색으로 BoundingBox를 표시하세요.

```

[5]: # TODO 4: 안전/위험 인원을 색상으로 구분하여 표시하세요
img = image.open(image_path)
draw = ImageDraw.Draw(img)
w, h = img.size

```

Simple 7 Python 3 (pykernel) Idle Initialized (additional servers needed) Instance MEM 83%

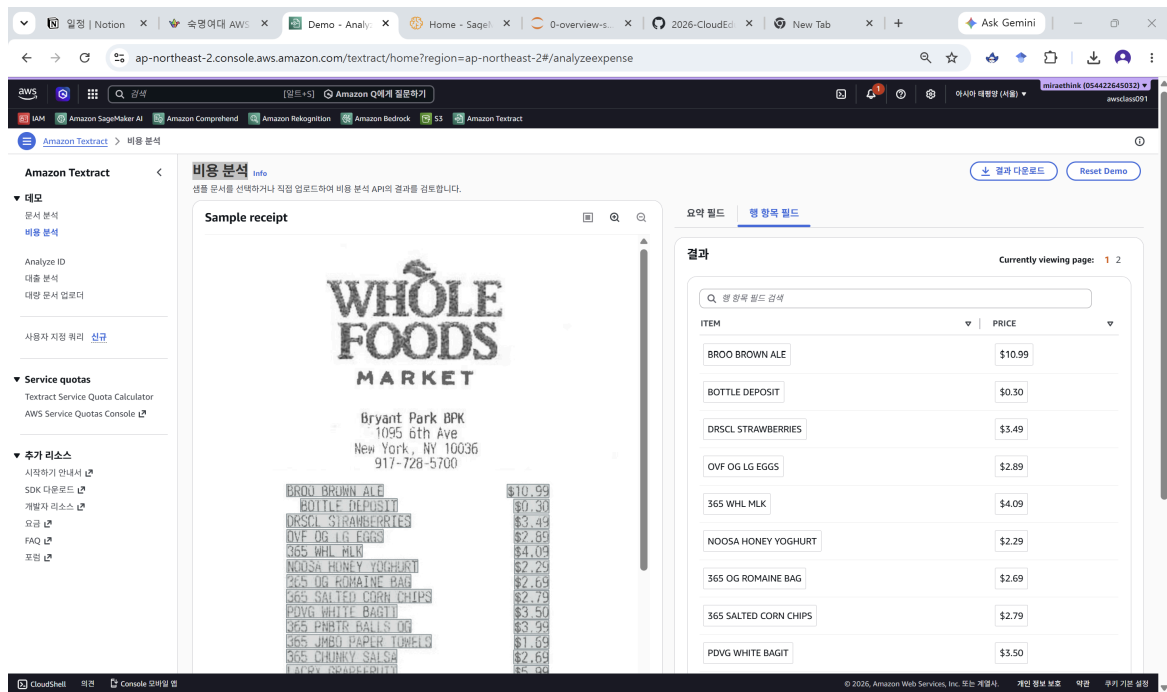


[실습] Textract 데모

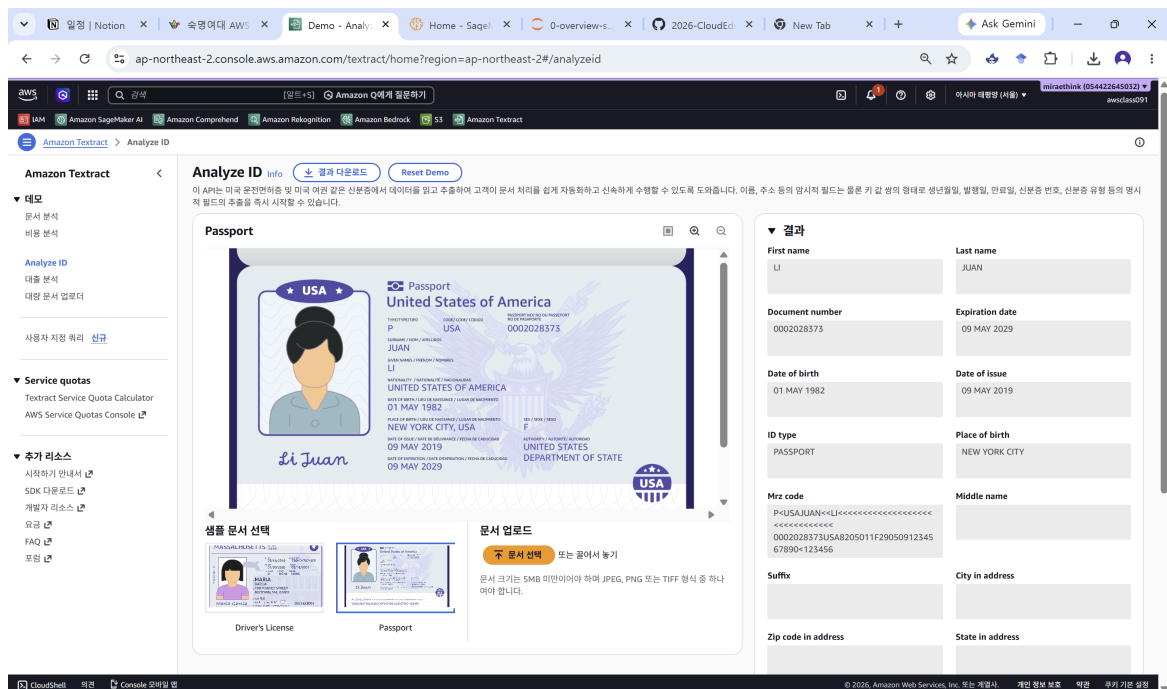
AWS console 접속

Amazon Textract 이동 → 데모

- 비용 분석



• Analyze ID



[실습] Textract API 실습

- Textract → 문서를 Block 단위로 분석 → Block 객체로 반환.

PAGE

LINE
WORD
WORD

```

└─ TABLE
  └─ CELL (row=1, col=1)
    └─ CELL (row=1, col=2)
  └─ KEY_VALUE_SET (KEY)
    └─ KEY_VALUE_SET (VALUE)
└─ QUERY
  └─ QUERY_RESULT

```

```

# Textract 클라이언트 초기화
textract = boto3.client('textract', region_name='ap-northeast-2')
s3 = boto3.client('s3', region_name='ap-northeast-2')

```

텍스트 감지 (Text Detection)

```

textract.detect_document_text(Document={'Bytes': doc_bytes})
- 문서에서 텍스트 추출

```

TODO 1: detect_document_text API 호출

문서에서 텍스트를 추출하세요.

```

[3]: # TODO 1: API를 호출하고 전체 Block 수를 출력하세요
response = textract.detect_document_text(
    Document={
        'Bytes': doc_bytes
    }
)

blocks = response['Blocks']
print(f"전체 Block 수: {len(blocks)}개")
print(f"페이지 수: {response['DocumentMetadata']['Pages']}페이지")

전체 Block 수: 4개
페이지 수: 1페이지

```

TODO 2: Block 타입별 개수 분석

반환된 Blocks에서 각 BlockType의 개수를 집계하세요.

```

[ ]: # TODO 2: BlockType별 개수를 집계하여 출력하세요
# 힌트: 딕셔너리를 사용해 각 타입의 개수를 세어보세요

type_counts = {} # 빈 딕셔너리 시작

```

```

for block in blocks:
    btype = block['BlockType']
    type_counts[btype] = type_counts.get(btype, 0) + 1

```

BlockType	설명
PAGE	전체 페이지, CHILD로 LINE들을 가짐
LINE	텍스트 한 줄, CHILD로 WORD들을 가짐

BlockType	설명
WORD	개별 단어

The screenshot shows a JupyterLab interface with a notebook titled '1-text-detection-exercise.ipynb'. The code in the notebook is as follows:

```
print(f"전체 Block 수: {len(blocks)}개")
print(f"페이지 수: {response['DocumentMetadata']['Pages']}페이지" ) # ← 'Pages'
```

전체 Block 수: 4개
페이지 수: 1페이지

TODO 2: Block 타입별 개수 분석

반환된 Blocks에서 각 BlockType의 개수를 집계하세요.

```
[4]: # TODO 2: BlockType별 개수를 집계하여 출력하세요
# 힌트: 딕셔너리를 사용해 각 타입의 개수를 세어보세요

type_counts = {} # 빈 딕셔너리 시작

for block in blocks:
    btype = block['BlockType'] # ← 'BlockType'
    type_counts[btype] = type_counts.get(btype, 0) + 1 # ← btype, 1

print("BlockType 분포:")
print("-" * 30)
for btype, count in sorted(type_counts.items()):
    print(f" {btype:<20} {count:>5}개")
```

BlockType 분포:

LINE	1개
PAGE	1개
WORD	2개

TODO 3: LINE 블록에서 텍스트 추출

문서 분석 — 품(양식) & 표(Table) 추출

```
blocks = response['Blocks']
block_map = {b['Id']: b for b in blocks} # Id → 블록 빠른 조회용
```

```
textract.analyze_document(
    Document={'Bytes': bytes},
    FeatureTypes=['FORMS', 'TABLES'] # 분석 기능 선택
)
```

2-forms-tables-exercise.ipynb

File Edit View Run Kernel Git Tabs Settings Help

2-forms-tables-exercise.ipynb

TODO 1: analyze_document API 호출 (FORMS + TABLES)

```
[2]: # TODO 1: FORMS와 TABLES 기능을 모두 활성화하여 API를 호출하세요
      # + API 메시지명 (analyze_document)
      response = textract.analyze_document(
          Documents={'Bytes': doc_bytes},
          FeatureTypes=['FORMS', 'TABLES']
      )

      blocks = response['Blocks']
      block_map = {b['Id']: b for b in blocks}

      # BlockType 분포 확인
      from collections import Counter
      type_counts = Counter(b['BlockType'] for b in blocks)
      print("BlockType 분포:")
      for btype, cnt in sorted(type_counts.items()):
          print(f"    {btype}: {cnt}>5개")
```

BlockType 분포:

CELL	35개
KEY_VALUE_SET	8개
LINE	28개
MERGED_CELL	2개
PAGE	1개
TABLE	2개
TABLE_TITLE	2개
WORD	63개

TODO 2: FORMS — Key-Value 쌍 추출

KEY_VALUE_SET 블록에서 키와 값 텍스트를 추출하는 함수를 완성하세요.

```
# FORMS: Key-Value 쌍 추출
def extract_forms(blocks, block_map):
    forms = {}
    key_blocks = [b for b in blocks
                   if b['BlockType'] == 'KEY_VALUE_SET'
                   and 'KEY' in b.get('EntityTypes', [])]

    for key_block in key_blocks:
        key_text = get_text_from_block(key_block, block_map)
        value_text = ''
        for rel in key_block.get('Relationships', []):
            if rel['Type'] == 'VALUE':
                for value_id in rel['Ids']:
                    value_block = block_map.get(value_id)
                    if value_block:
                        value_text = get_text_from_block(value_block, block_map)

        if key_text:
            forms[key_text] = value_text
    return forms
```

The screenshot shows a Jupyter Notebook titled '2-forms-tables-exercise.ipynb'. The code defines a function to extract data from a document structure. It processes relationships and extracts text from blocks. The output shows 4 extracted key-value pairs:

```
추출된 Key-Value 쌍: 4개
```

폼 데이터:

Full Name:	Jane Doe
Home Address:	123 Any Street, Any Town, USA
Mailing Address:	same as home address
Phone Number:	555-8100

TODO 3: TABLES — 표 데이터를 DataFrame으로 변환

TABLE 블록을 파싱하여 pandas DataFrame으로 변환하세요.

```
# TABLES: 표 → DataFrame 변환
def extract_tables(blocks, block_map):
    tables = []
    for table_block in [b for b in blocks if b['BlockType'] == 'TABLE']:
        rows = {}
        for rel in table_block.get('Relationships', []):
            if rel['Type'] == 'CHILD':
                for cell_id in rel['Ids']:
                    cell = block_map.get(cell_id)
                    if cell and cell['BlockType'] == 'CELL':
                        r, c = cell['RowIndex'], cell['ColumnIndex']
                        rows.setdefault(r, {})[c] = get_text_from_block(cell, block_map)

        if rows:
            max_r = max(rows.keys())
            max_c = max(max(r.keys()) for r in rows.values())
            data = [[rows.get(r, {}).get(c, '') for c in range(1, max_c+1)]
                    for r in range(1, max_r+1)]
            df = pd.DataFrame(data)
            df.columns = df.iloc[0] # 첫 행 → 헤더
            tables.append(df[1:].reset_index(drop=True))
    return tables
```

The screenshot shows a JupyterLab environment with a file explorer on the left, a code editor in the center, and a table visualization at the bottom. The code in the notebook processes data from a CSV file and appends it to a table. The table visualization shows two tables: 'Table #1' and 'Table #2'.

```
data.append(row_data)

df = pd.DataFrame(data)
# 첫 행을 헤더로 설정
df.columns = df.iloc[0]
df = df[1:].reset_index(drop=True)
tables.append(df)

return tables

table_list = extract_tables(blocks, block_map)
print(f"검지된 표: {len(table_list)}개")
for i, df in enumerate(table_list):
    print(f"\n표 #{i+1} ({df.shape[0]}행 x {df.shape[1]}열):")
    print(df.to_string())
```

검지된 표: 2개

표 #1 (4행 x 2열):

	Applicant	Information
0	Full Name: Jane	Doe
1	Phone Number:	555-0100
2	Home Address:	123 Any Street, Any Town, USA
3	Mailing Address:	same as home address

표 #2 (4행 x 5열):

	Start Date	End Date	Employer Name	Position Held	Reason for leaving
0	1/15/2009	6/30/2011	Any Company	Assistant Baker	Family relocated
1	7/1/2011	8/10/2013	Best Corp.	Baker	Better opportunity
2	8/15/2013	present	Example Corp.	Head Baker	N/A, current employer

TODO 4: TABLE과 KEY_VALUE 위치 시각화

TABLE은 빨간색, KEY_VALUE_SET(KEY)는 초록색으로 표시하세요.

블록 타입별 시각화

```
color_map = {'TABLE': 'red', 'CELL': 'orange', 'KEY_VALUE_SET': 'green'}
```

```
for block in blocks:
```

```
    if block['BlockType'] not in color_map or 'Geometry' not in block:
        continue
```

```
    box = block['Geometry']['BoundingBox']
```

```
    left, top = box['Left'] * w, box['Top'] * h
```

```
    right, bottom = left + box['Width'] * w, top + box['Height'] * h
```

```
    draw.rectangle([left, top, right, bottom],
```

```
                    outline=color_map[block['BlockType']],
```

```
                    width=3 if block['BlockType'] == 'TABLE' else 1)
```


The screenshot shows a JupyterLab environment with a file explorer on the left and a notebook in the center. The notebook contains an 'Employment Application' form and a table of 'Previous Employment History'.

Employment Application

Applicant Information

Full Name:	Jane Doe
Phone Number:	555-0100
Home Address:	123 Any Street, Any Town, USA
Mailing Address:	same as home address

Previous Employment History

Start Date	End Date	Employer Name	Position Held	Reason for leaving
1/15/2009	6/30/2011	Any Company	Assistant Baker	Family relocated
7/1/2011	8/10/2013	Best Corp.	Baker	Better opportunity
8/15/2013	present	Example Corp.	Head Baker	N/A, current employer

쿼리 기반 정보 추출 (Queries)

QUERY Block 구조

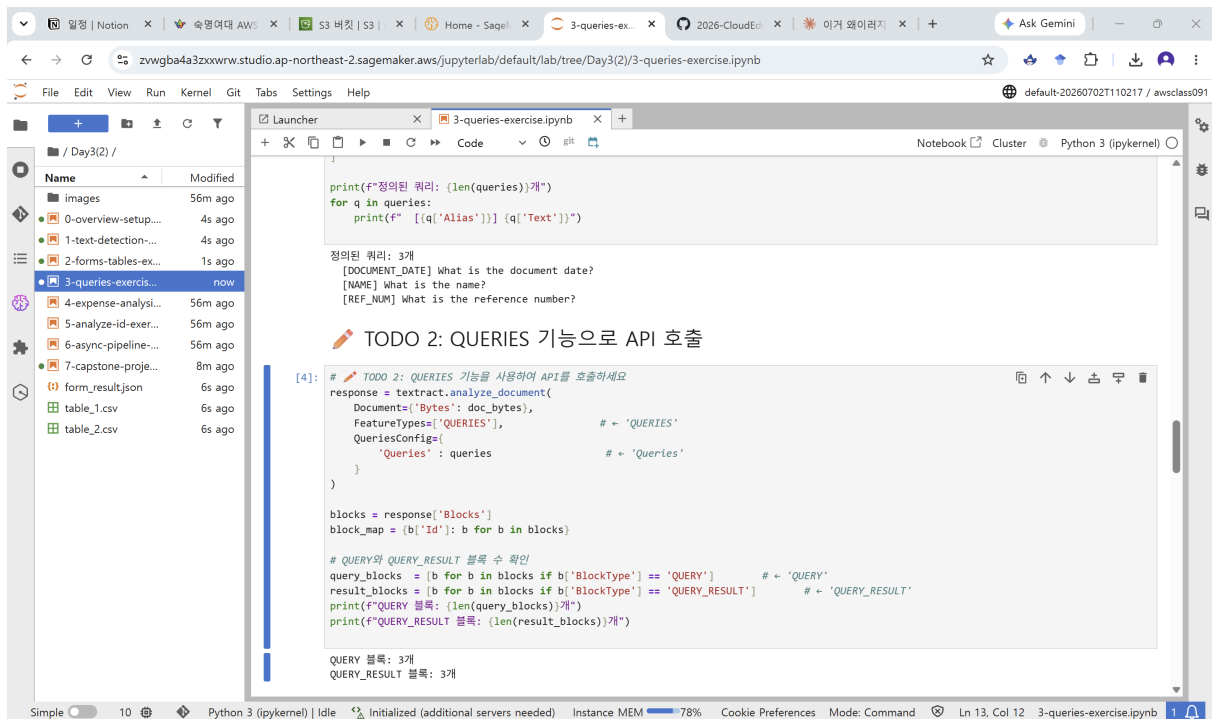
QUERY Block

- └ Text: '질문 텍스트'
- └ Query.Alias: '별칭'
- └ Relationships[Type='ANSWER'] → QUERY_RESULT Block
 - └ Text: '답변 텍스트'
 - └ Confidence: 신뢰도

쿼리 정의

```
queries = [
    {'Text': ' ', 'Alias': ' '}, # 질문 텍스트, 별칭 쌍으로
    {'Text': ' ', 'Alias': ' '},
    {'Text': ' ', 'Alias': ' '},
    # 문서 유형에 맞게 추가
]

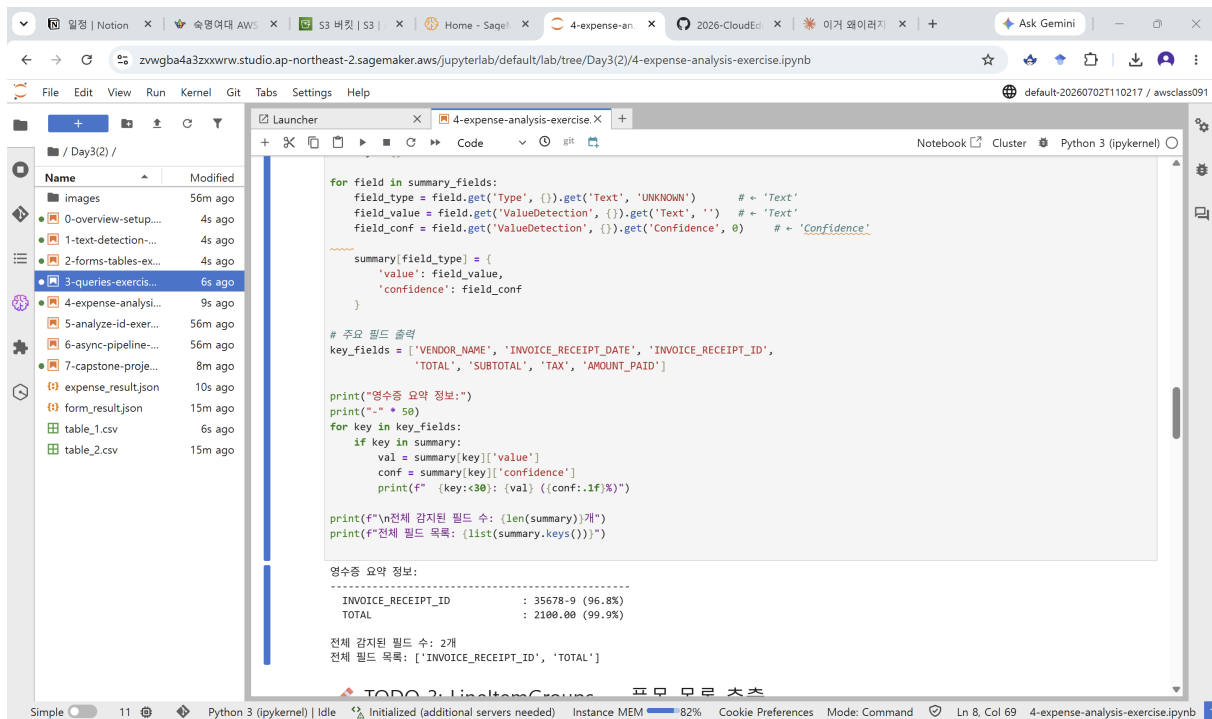
textract.analyze_document(
    Document={'Bytes': bytes},
    FeatureTypes=['QUERIES'],
    QueriesConfig={'Queries': queries}
)
```



영수증 / 인보이스 분석 (Expense Analysis)

```
# 응답 구조
response
├── ExpenseDocuments[]
│   ├── SummaryFields[]
│   │   ├── Type.Text          ← 헤더 정보 (벤더명, 날짜, 총액 등)
│   │   ├── ValueDetection.Text ← 필드 유형 (VENDOR_NAME, TOTAL, DATE 등)
│   │   └── Confidence          ← 추출된 값
│   └── LineItemGroups[]
│       ├── LineItems[]
│       │   ├── LineItemExpenseFields[]
│       │   │   ├── Type.Text    ← 품목 그룹
│       │   │   └── ValueDetection.Text ← 개별 품목
│       │   └── LineItemExpenseFields[]
│       │       ├── Type.Text    ← ITEM, QUANTITY, PRICE, UNIT_PRICE 등
│       │       └── ValueDetection.Text
```

```
textextract.analyze_expense(Document={'Bytes': bytes})
```

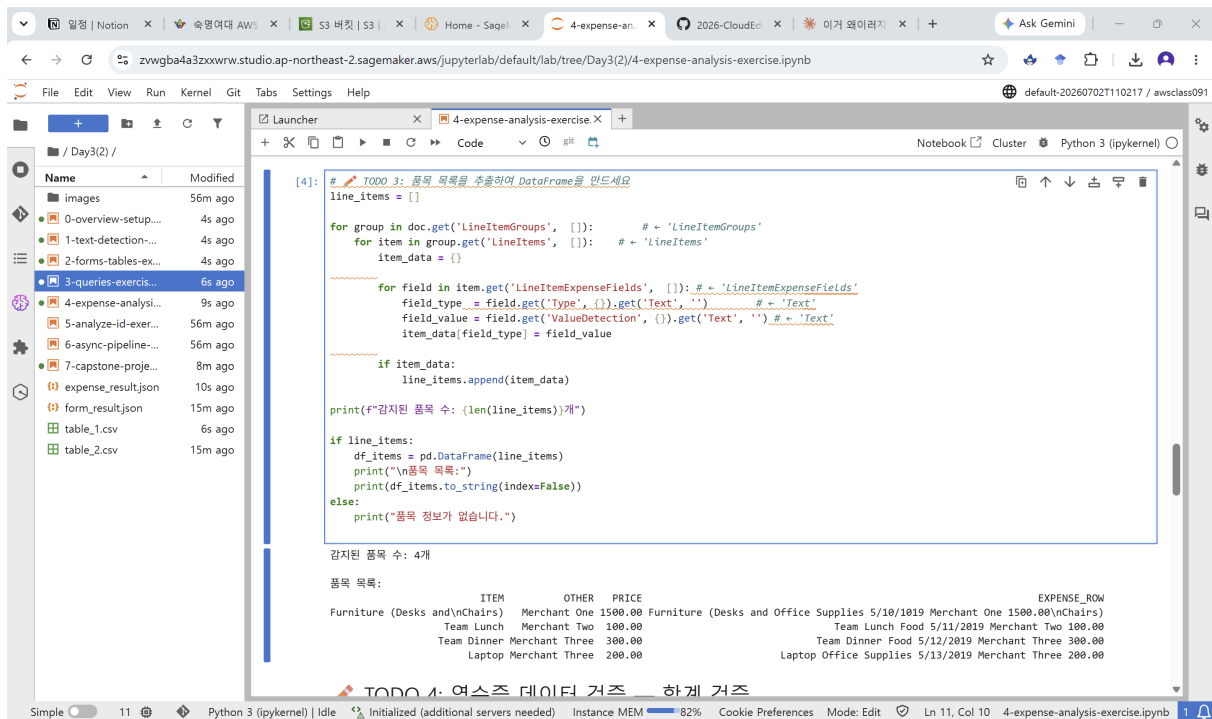
```

# LineItemGroups: 품목 목록 → DataFrame
line_items = []

for group in doc.get('LineItemGroups', []):
    for item in group.get('LineItems', []):
        item_data = {}
        for field in item.get('LineItemExpenseFields', []):
            field_type = field.get('Type', {}).get('Text', '')
            field_value = field.get('ValueDetection', {}).get('Text', '')
            item_data[field_type] = field_value
        if item_data:
            line_items.append(item_data)

df_items = pd.DataFrame(line_items)
# 컬럼 예: ITEM / QUANTITY / UNIT_PRICE / PRICE

```



신분증 분석 (Identity Document Analysis)

```
# 응답 구조
response
├── IdentityDocuments[]
│   └── IdentityDocumentFields[]
│       ├── Type
│       │   └── Text ← 필드명 (FIRST_NAME, LAST_NAME, DATE_OF_BIRT
H 등)
│       ├── NormalizedValue.Value ← 정규화된 타입
│       └── ValueDetection
│           ├── Text ← 추출된 원본 텍스트
│           ├── Confidence ← 신뢰도
│           └── NormalizedValue.Value ← 정규화된 값 (날짜: YYYY-MM-DD 형식
등)
```

```
textract.analyze_id(DocumentPages=[{'Bytes': bytes}])
```

Launcher

2nd Dose Vaccine A	Pfizer BB5678	2/8/2021	mm dd yy
Booster Shot Vaccine A			mm dd yy
Other			mm dd yy

TODO 1: analyze_id API 호출

analyze_id 는 'Document' 대신 'DocumentPages' (리스트)를 사용합니다.

```
[4]: # TODO 1: analyze_id API를 호출하세요
# 주의: Document가 아닌 DocumentPages (리스트)를 사용!
response = texttract.analyze_id(
    DocumentPages[
        {'Bytes': doc_bytes}
    ]
)

identity_docs = response['IdentityDocuments']
print(f"감지된 신분증 수: {len(identity_docs)}개")

id_doc = identity_docs[0]
fields = id_doc.get('IdentityDocumentFields', [])
print(f"감지된 필드 수: {len(fields)}개")

감지된 신분증 수: 1개
감지된 필드 수: 21개
```

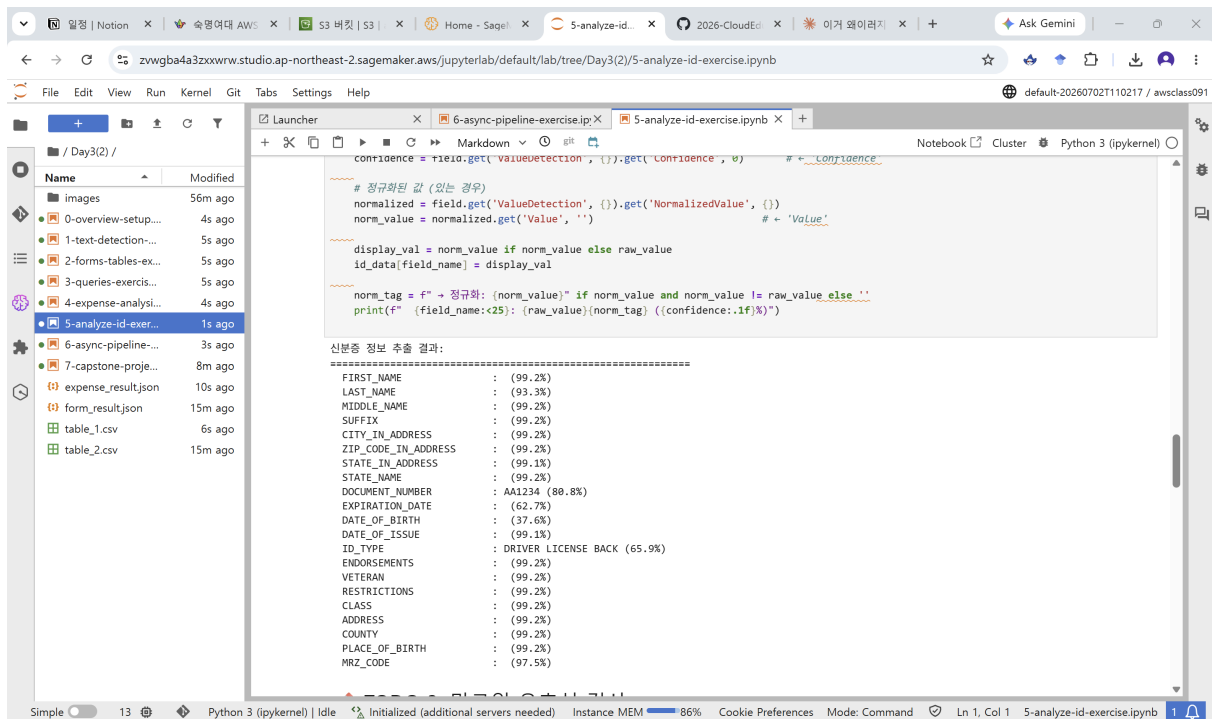
타입	설명	정규화 예시
FIRST_NAME	이름	-
LAST_NAME	성	-
DATE_OF_BIRTH	생년월일	YYYY-MM-DD
EXPIRATION_DATE	만료일	YYYY-MM-DD
DOCUMENT_NUMBER	문서 번호	-
ID_TYPE	문서 유형	PASSPORT, DRIVER_LICENSE
STATE_IN_ADDRESS	주/지역	-
MRZ_CODE	기계 판독 영역	-

```
# 모든 필드 파싱 (원본값 + 정규화값)
id_data = {}

for field in fields:
    field_name = field.get('Type', {}).get('Text', 'UNKNOWN')
    raw_value = field.get('ValueDetection', {}).get('Text', '')
    confidence = field.get('ValueDetection', {}).get('Confidence', 0)

    # 정규화 값 (날짜 → YYYY-MM-DD 등)
    norm_value = field.get('ValueDetection', {}).get('NormalizedValue',
    {}).get('Value', '')

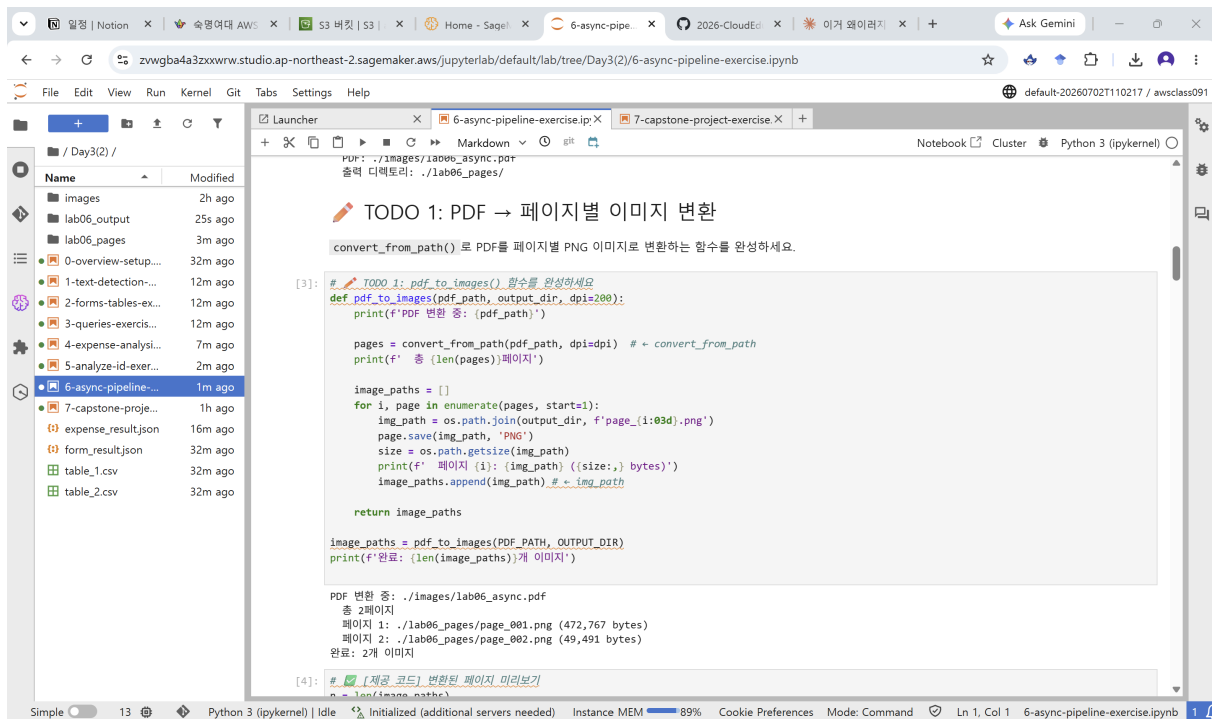
    id_data[field_name] = norm_value if norm_value else raw_value
```



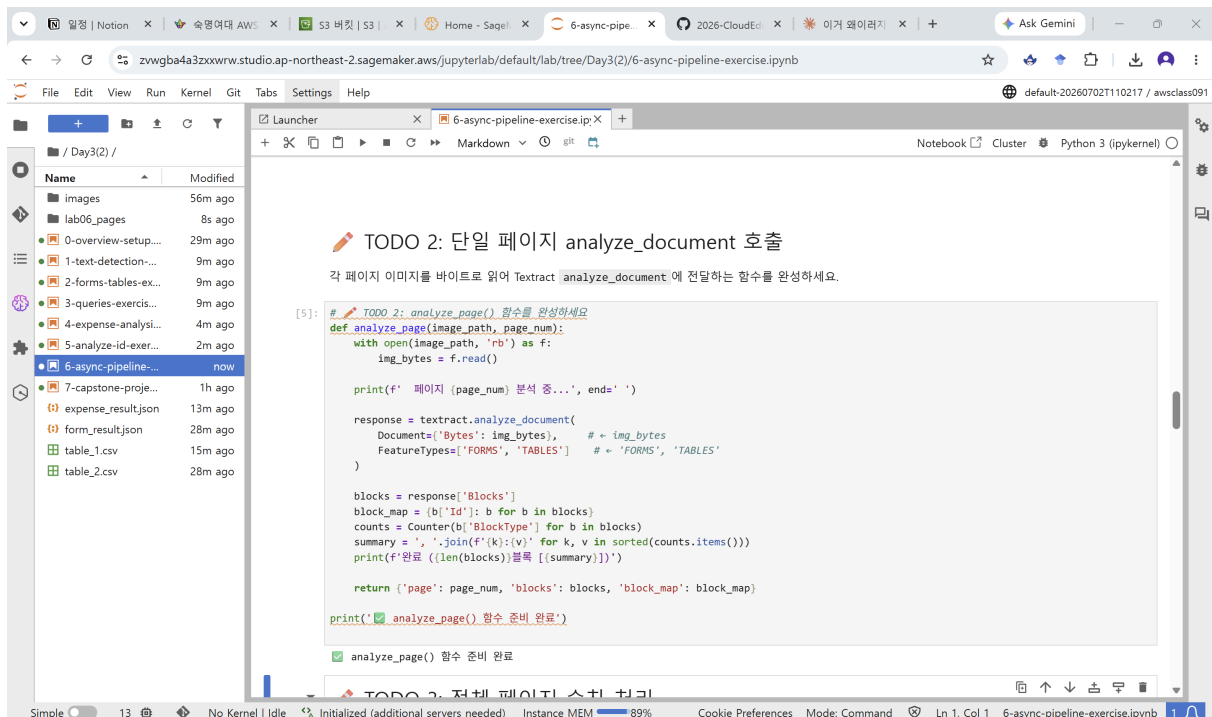
다중 페이지 PDF 비동기 처리 파이프라인

```
# PDF → 페이지별 이미지 변환
from pdf2image import convert_from_path

def pdf_to_images(pdf_path, output_dir, dpi=200):
    pages = convert_from_path(pdf_path, dpi=dpi)
    image_paths = []
    for i, page in enumerate(pages, start=1):
        img_path = os.path.join(output_dir, f'page_{i:03d}.png')
        page.save(img_path, 'PNG')
        image_paths.append(img_path)
    return image_paths
```



```
textextract.analyze_document(
    Document={'Bytes': bytes},
    FeatureTypes=['FORMS', 'TABLES'] # 분석 기능 선택
)
```



```
# 페이지별 텍스트(LINE) 추출
def extract_text_by_page(all_results):
```



```

page_texts = {}
for result in all_results:
    lines = [b.get('Text', '') for b in result['blocks']]
    if b['BlockType'] == 'LINE'
    page_texts[result['page']] = lines
return page_texts

```

The screenshot shows a Jupyter Notebook environment. On the left, a file explorer displays a directory structure with files like 'images', 'lab06_output', 'lab06_pages', and various exercise files. The main area shows a code cell with the following Python code:

```

[8]: # TODO 4: 각 페이지에서 LINE 텍스트를 추출하는 함수를 완성하세요
def extract_text_by_page(all_results):
    page_texts = {}
    for result in all_results:
        page_num = result['page']
        lines = [
            b.get('Text', '')
            for b in result['blocks']
            if b['BlockType'] == 'LINE' # <- 'BlockType', 'LINE'
        ]
        page_texts[page_num] = lines
    return page_texts

page_texts = extract_text_by_page(all_results)
print('페이지별 텍스트:')
for pn, lines in page_texts.items():
    print(f'페이지 {pn}: {len(lines)}줄')
    for line in lines[:5]:
        print(f'    {line}')

페이지별 텍스트:
페이지 1: 36줄
Amazon Texttract
Amazon Texttract is a service that automatically extracts text and data from scanned
documents. Amazon Texttract goes beyond simple optical character recognition (OCR) to
also identify the contents of fields in forms and information stored in tables.
Many companies today extract data from documents and forms through manual data
페이지 2: 3줄
a workflow that automatically redacts personally identifiable information (PII) for their review
before archiving claim forms by automatically recognizing the important key-value pairs that
require protection.

```

```

# 페이지별 폼(Key-Value) 추출
def extract_forms_by_page(all_results):
    page_forms = {}
    for result in all_results:
        blocks, block_map = result['blocks'], result['block_map']
        forms = {}
        key_blocks = [b for b in blocks
                        if b['BlockType'] == 'KEY_VALUE_SET'
                        and 'KEY' in b.get('EntityTypes', [])]
        for key_block in key_blocks:
            key_text = get_text_from_block(key_block, block_map)
            value_text = ''
            for rel in key_block.get('Relationships', []):
                if rel['Type'] == 'VALUE':
                    for vid in rel['Ids']:
                        vb = block_map.get(vid)
                        if vb:
                            value_text = get_text_from_block(vb, block_ma
p)

            if key_text:

```

```
forms[key_text] = value_text
page_forms[result['page']] = forms
return page_forms
```

```
for key_block in key_blocks:
    key_text = get_text_from_block(key_block, block_map)
    value_text = ''
    for rel in key_block.get('Relationships', []):
        if rel['Type'] == 'VALUE': # ← 'Type', 'VALUE'
            for vid in rel['Ids']: # ← 'Ids'
                vb = block_map.get(vid)
                if vb:
                    value_text = get_text_from_block(vb, block_map) # ← vb

    if key_text:
        forms[key_text] = value_text
        page_forms[result['page']] = forms
    return page_forms

page_forms = extract_forms_by_page(all_results)
for pn, forms in page_forms.items():
    print(f'페이지 {pn}: {len(forms)}개 Key-Value')
    for k, v in list(forms.items())[:4]:
        print(f'  {k:<30}: {v}')

페이지 1: 3개 Key-Value
Maintain compliance in document archives: Because Amazon Textract identifies data types and form labels automatically, it's easy to maintain compliance with information controls. For example, an insurer could use Amazon Textract to feed Build automated document processing workflows. Amazon Textract can provide the inputs required to automatically process forms without human intervention. For example, a bank could write code to read PDFs of loan applications. The information contained in the document could be used to initiate all of the necessary background and credit checks to approve the loan so that customers can get instant results of their application rather than having to wait several days for manual review and validation.
Create smart search indexes : Extract structured data from documents and create a smart index using Amazon Elasticsearch Service to allow you to search through millions of financial statements quickly. For example, a mortgage company could use Amazon Textract to process millions of scanned loan applications in a matter of hours and have the extracted data indexed in Amazon Elasticsearch. This would allow them to create search experiences like "search for loan applications where applicant name is John Doe," or "search contracts where the interest rate is 2 percent."
페이지 2: 2개 Key-Value
a workflow that automatically redacts personally identifiable information (PII) for their review before archiving claim forms by automatically recognizing the important key-value pairs that require protection.:
```

```
# 페이지별 표(TABLE) → DataFrame
def extract_tables_by_page(all_results):
    page_tables = {}
    for result in all_results:
        blocks, block_map = result['blocks'], result['block_map']
        dfs = []
        for tb in [b for b in blocks if b['BlockType'] == 'TABLE']:
            rows = {}
            for rel in tb.get('Relationships', []):
                if rel['Type'] == 'CHILD':
                    for cid in rel['Ids']:
                        cell = block_map.get(cid)
                        if cell and cell['BlockType'] == 'CELL':
                            r, c = cell['RowIndex'], cell['ColumnIndex']
                            rows.setdefault(r, {})[c] = get_text_from_block(
                                cell, block_map)
            if rows:
                max_r = max(rows.keys())
                max_c = max(max(row.keys() for row in rows.values()))
                data = [[rows.get(r, {}).get(c, '') for c in range(1, max_c+1)]
                        for r in range(1, max_r+1)]
                df = pd.DataFrame(data)
```

```

df.columns = df.iloc[0]
dfs.append(df[1:].reset_index(drop=True))
page_tables[result['page']] = dfs
return page_tables

```

The screenshot shows a JupyterLab environment. On the left is a file explorer with a tree view of files and folders. The main area is a code editor with a Python script. The script processes table blocks, extracts relationships, and appends data to a DataFrame. The status bar at the bottom indicates the current state of the environment.

```

table_blocks = [b for b in blocks if b['BlockType'] == 'TABLE'] # + 'BlockType',...TABLE'

for tb in table_blocks:
    rows = {}
    for rel in tb.get('Relationships', []):
        if rel['Type'] == 'CHILD': # + 'Type'
            for cid in rel['Ids']: # + 'Ids'
                cell = block_map.get(cid)
                if cell and cell['BlockType'] == 'CELL':
                    r = cell['RowIndex'] # + 'RowIndex'
                    c = cell['ColumnIndex'] # + 'ColumnIndex'
                    rows.setdefault(r, {})[c] = get_text_from_block(cell, block_map)

    if rows:
        max_r = max(rows.keys())
        max_c = max(max(row.keys()) for row in rows.values())
        data = [[rows.get(r, {}).get(c, '') for c in range(1, max_c+1)] for r in range(1, max_r+1)]
        df = pd.DataFrame(data)
        if not df.empty:
            df.columns = df.iloc[0]
            df = df[1:].reset_index(drop=True)
            dfs.append(df)

    page_tables[page_num] = dfs
    return page_tables

page_tables = extract_tables_by_page(all_results)
for pn, dfs in page_tables.items():
    print(f'페이지 {pn}: {len(dfs)}개 표')
    for i, df in enumerate(dfs):
        print(f'  표 #{i+1}: {(df.shape[0])}행 x {(df.shape[1])}열')
        print(df.head(3).to_string())

페이지 1: 0개 표
페이지 2: 0개 표

```

Simple 13 Python 3 (ipykernel) | Idle Initialized (additional servers needed) Instance MEM 89% Cookie Preferences Mode: Command Ln 1, Col 1 6-async-pipeline-exercise.ipynb 1